MDTS4214_Assignment 8_710

PROTYAY CHATTERJEE

2026-03-30

PREDICTIVE ANALYTICS

Problem Set 4

1. Problem to demonstrate the fitting of a binary response variable using logistic regression

a. Consider the Weekly data available in the ISLR package of R. Use the data from 1990 to 2008 as train data and the data on 2009-2010 as test data.

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.5.2

## Weekly - The data set involved
head(Weekly)

##   Year   Lag1   Lag2   Lag3   Lag4   Lag5    Volume  Today Direction
## 1 1990  0.816  1.572 -3.936 -0.229 -3.484 0.1549760 -0.270      Down
## 2 1990 -0.270  0.816  1.572 -3.936 -0.229 0.1485740 -2.576      Down
## 3 1990 -2.576 -0.270  0.816  1.572 -3.936 0.1598375  3.514        Up
## 4 1990  3.514 -2.576 -0.270  0.816  1.572 0.1616300  0.712        Up
## 5 1990  0.712  3.514 -2.576 -0.270  0.816 0.1537280  1.178        Up
## 6 1990  1.178  0.712  3.514 -2.576 -0.270 0.1544440 -1.372      Down

train_data = Weekly[Weekly$Year <= 2008, ]
test_data = Weekly[Weekly$Year > 2008, ]
```

b. Using the training data, fit a logistic regression with Direction as the response and the five lag variables plus Volume as predictors. Are there any significant predictors? If not, which of the tests is reporting the least p value. Interpret the coefficient corresponding to that test.

```
logit_model = glm(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
                  data = train_data, family = "binomial")
summary(logit_model)

##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##     Volume, family = "binomial", data = train_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.33258    0.09421   3.530 0.000415 ***
## Lag1        -0.06231    0.02935  -2.123 0.033762 *
## Lag2         0.04468    0.02982   1.499 0.134002
## Lag3        -0.01546    0.02948  -0.524 0.599933
```

```
## Lag4        -0.03111    0.02924  -1.064 0.287241
## Lag5        -0.03775    0.02924  -1.291 0.196774
## Volume      -0.08972    0.05410  -1.658 0.097240 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1354.7  on 984  degrees of freedom
## Residual deviance: 1342.3  on 978  degrees of freedom
## AIC: 1356.3
##
## Number of Fisher Scoring iterations: 4
```

At a 5% significance level ($\alpha = 0.05$), there is one significant predictor: Lag1, as its p-value (0.033762) is less than 0.05.The predictor with the least p-value is Lag1 ($p \approx 0.034$).

The coefficient for Lag1 is approximately -0.06231.

Log - Odds : For every one-unit increase in the percentage return from last week, the log-odds of the market going "Up" decreases by 0.06231, holding all other variables (Lag2 through Lag5 and Volume) constant.

Direction : Since the coefficient is negative, an increase in the percentage return from the previous week (Lag1) is associated with a decrease in the probability of the market going "Up" this week.

Odds Ratio: To make this more intuitive, we can exponentiate the coefficient ($e^{-0.06231} \approx 0.94$). This means a one-unit increase in Lag1 results in the odds of an "Up" direction being multiplied by 0.94 (a 6% decrease in odds).

c. Estimate the odds of market going "up" at median values of the predictors.

```r
medians = data.frame(
  Lag1 = median(train_data$Lag1),
  Lag2 = median(train_data$Lag2),
  Lag3 = median(train_data$Lag3),
  Lag4 = median(train_data$Lag4),
  Lag5 = median(train_data$Lag5),
  Volume = median(train_data$Volume)
)

prob_up = predict(logit_model, medians, type = "response")
odds_up = prob_up / (1 - prob_up)
odds_up
```

```
##        1
## 1.267475
```

Since the odds are greater than 1, it indicates that the market is more likely to go “Up” than “Down” when the predictors are at their median values.

The Ratio: The odds of $1.267:1$ mean that for every 100 times the market goes “Down” under these conditions, it is expected to go “Up” about 127 times.

e. Using the fitted model and the predictor values from the test data, find an optimal cut point that minimizes T P R(1 − F P R). Report the confusion matrix corresponding to the optimal cut-point value. From the confusion matrix compute the test error rate.

```
probs_test = predict(logit_model, test_data, type = "response")
actual_test = test_data$Direction

thresholds = seq(0, 1, length = 100)
results = sapply(thresholds, function(t) {
  preds = ifelse(probs_test > t, "Up", "Down")
  tp = sum(preds == "Up" & actual_test == "Up")
  tn = sum(preds == "Down" & actual_test == "Down")
  fp = sum(preds == "Up" & actual_test == "Down")
  fn = sum(preds == "Down" & actual_test == "Up")

  tpr = tp / (tp + fn)
  fpr = fp / (fp + tn)
  return(tpr * (1 - fpr))
})

opt_t = thresholds[which.max(results)]
opt_t

## [1] 0.4545455

# Confusion Matrix
final_preds = ifelse(probs_test > opt_t, "Up", "Down")
conf_matrix = table(Predicted = final_preds, Actual = actual_test)
conf_matrix

##          Actual
## Predicted Down Up
##      Down   21 27
##      Up     22 34

# Test Error Rate
error_rate = mean(final_preds != actual_test)
error_rate

## [1] 0.4711538
```

f. Is there any change in the test error rate if we have worked with only the two predictors lag 1 and lag 2 ?

```
reduced_model <- glm(Direction ~ Lag1 + Lag2, data = train_data, family = "bi
nomial")
```

```r
probs_reduced <- predict(reduced_model, test_data, type = "response")

preds_reduced <- ifelse(probs_reduced > 0.5, "Up", "Down")
error_rate_reduced <- mean(preds_reduced != actual_test)

# Compare with original model error rate
preds_orig_50 <- ifelse(probs_test > 0.5, "Up", "Down")
error_orig_50 <- mean(preds_orig_50 != actual_test)

list(Original_Error_50 = error_orig_50, Reduced_Error_50 = error_rate_reduced)
```

```
## $Original_Error_50
## [1] 0.5384615
##
## $Reduced_Error_50
## [1] 0.4230769
```

We see that the test error rate decreases .